SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.* *(BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I M.Niharika(192365019) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Niharika

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

1.Given the three-address code sequence:
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
Simulate the peephole optimization process on this sequence. Produce the optimized
instruction sequence and justify each optimization step applied, including algebraic
simplification, constant folding, and dead code elimination.

1. Peephole Optimization

Given:
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y

Step 1: Algebraic simplification

- x * 1 → x
- t2 - 0 → t2

So:
t1 = x
t2 = t1 + y
t3 = t2
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y

Step 2: Constant folding
Since:
t3 * 0 = 0
Therefore:
t4 = 0
The condition becomes:
if 0 != 0 goto L1
This condition is always **false**, so the jump can never occur.
Step 3: Dead code elimination
Since t4 is always zero and the condition is false:
if t4 != 0 goto L1
can be removed.
Also, t1 and t3 are only temporary copies, so they can be eliminated by copy propagation.
Optimized sequence
t2 = x + y
t5 = y + y
Optimizations applied:

1. x * 1 → x — algebraic simplification.
2. t2 - 0 → t2 — algebraic simplification.
3. t3 * 0 → 0 — constant folding.
4. if 0 != 0 is always false — conditional simplification.
5. Unreachable/dead instructions are eliminated.
6. Temporary copies are removed.

2.Partition the following intermediate code into basic blocks and construct the corresponding
control flow graph. Identify all leader statements and mention the rules used to identify them:
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
Given:
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
Step 1: Identify leaders
Rules for identifying leaders:

1. The **first statement** is a leader.
2. The **target of a conditional/unconditional jump** is a leader.
3. The statement **immediately following a jump** is a leader.

Therefore:

- Statement **1** → first statement
- Statement **6** → target of goto 6
- Statement **4** → follows conditional jump at 3
- Statement **7** → target of goto 7

Leaders

1, 4, 6, 7

Basic Blocks

B1:

(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6

B2:

(4) z = y - d
(5) goto 7

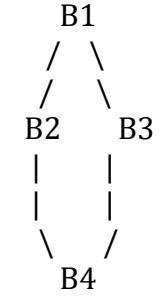
B3:

(6) z = y + d

B4:

(7) w = z * 2

Control Flow Graph



More specifically:

B1 → B3 (if y > 50)
B1 → B2 (otherwise)
B2 → B4
B3 → B4

3. Construct the DAG for the following basic block:

p = a + b
q = a + b
r = p * q
s = a + b
t = r + s

Using the DAG, identify all common sub-expressions. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

DAG Construction and Common Sub-expressions

Given:

p = a + b
q = a + b
r = p * q
s = a + b
t = r + s

DAG

The expression a + b occurs three times.

Create one common node:

```
    (+)
   /   \
  a     b
```

This node is used by p, q, and s.

Then:

```
    (*)
   /   \
 p/q   p/q
```

Finally:

```
    (+)
   /   \
  r     s
```

Common sub-expression

a + b

occurs in:

p = a + b

q = a + b

s = a + b

Therefore, compute it only once.

Optimized code

t1 = a + b

r = t1 * t1

t = r + t1

Simple target code

```
MOV R0, a
ADD R0, b      ; R0 = a+b

MOV R1, R0
MUL R1, R0     ; R1 = (a+b)*(a+b)

ADD R1, R0     ; R1 = r+s

MOV t, R1
```

Register allocation

- R0 stores the common expression a+b.
- R1 stores the multiplication result.
- R0 is retained because it is needed again for t = r+s.
- Finally, the result is stored into t.

4. For a target machine with two registers R0 and R1, simulate the working of a simple code generator for the code:

t1 = a + b

t2 = c * d

t3 = t1 - t2

t4 = e + f

t5 = t3 / t4

Show the register descriptor and address descriptor after each instruction is processed.

Two-Register Code Generation

Given:
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
Registers: **R0, R1**

Initial state

Register Descriptor:
R0 = empty
R1 = empty

Address Descriptor:
a = memory
b = memory
c = memory
d = memory
e = memory
f = memory

Instruction 1: t1 = a + b

LOAD R0, a
ADD R0, b
State:
R0 → t1
R1 → empty
Address descriptor:
t1 → R0

Instruction 2: t2 = c * d

LOAD R1, c
MUL R1, d
State:
R0 → t1
R1 → t2
Address descriptor:
t1 → R0
t2 → R1

Instruction 3: t3 = t1 - t2

SUB R0, R1
Now:
R0 → t3
R1 → t2
t2 is no longer required, so R1 becomes available.
Address descriptor:
t3 → R0

Instruction 4: t4 = e + f
LOAD R1, e
ADD R1, f
State:
R0 → t3
R1 → t4
Address descriptor:
t3 → R0
t4 → R1

Instruction 5: t5 = t3 / t4
DIV R0, R1
Final state:
R0 → t5
R1 → t4
Final result:
t5 = (a+b-c*d)/(e+f)
Register descriptor summary

Step	R0	R1
Initial	Empty	Empty
t1	t1	Empty
t2	t1	t2
t3	t3	t2
t4	t3	t4
t5	t5	t4

5. For a RISC-style target machine with instructions LOAD, STORE, ADD, SUB, MUL, MOV, simulate target code generation for the expression:
(a - b) * (c + d) - e
Show the complete sequence of target instructions generated. Also explain the runtime
storage management and stack frame layout used during the computation.
Given the three-address code sequence:
t1 = a + b,

t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3
RISC Target Code Generation
Expression:
(a - b) * (c + d) - e
Step 1: Calculate (a-b)
LOAD R0, a
SUB R0, b
Now:
R0 = a - b

Step 2: Calculate (c+d)
LOAD R1, c
ADD R1, d
Now:
R1 = c + d
Step 3: Multiply
MUL R0, R1
Now:
R0 = (a-b) * (c+d)
Step 4: Subtract e
SUB R0, e
Final:
R0 = (a-b)*(c+d)-e
Complete target code

```
LOAD R0, a
SUB  R0, b

LOAD R1, c
ADD  R1, d

MUL  R0, R1
SUB  R0, e

STORE result, R0
```

Runtime storage management
During execution:

- Variables a, b, c, d, e are stored in memory.
- Intermediate values are temporarily stored in registers.
- R0 contains (a-b) and later the final result.
- R1 contains (c+d).
- STORE saves the final result into memory.

Stack frame layout
A simple procedure stack frame can contain:

```
+------------------+
| Return Address   |
+------------------+
| Saved Registers  |
+------------------+
| Local Variables  |
+------------------+
| Temporaries      |
+------------------+
| Parameters       |
+------------------+
```

For this expression, the compiler mainly uses **R0 and R1**, so no large temporary stack storage is required.

Additional Three-Address Code
Given:

t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3

Optimization

1. Algebraic simplification

t2 = t1 + 0
becomes:
t2 = t1
And:
t3 = t2 * 1
becomes:
t3 = t2
Using copy propagation:
t3 = a + b
Therefore:
if a + b == 0 goto L1
and:
t4 = (a+b) + (a+b)

Optimized three-address code

t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1
This removes the unnecessary +0, *1, and temporary assignments.

Annexure A

SIMULATION TASK

QUESTIONS

1.Given the three-address code sequence:
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2.Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2

---

3. Construct the **DAG** for the following basic block:
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

---

4. For a target machine with two registers **R0 and R1**, simulate the working of a **simple code generator** for the code:
t1 = a + b
t2 = c * d
t3 = t1 - t2

t4 = e + f
t5 = t3 / t4
Show the **register descriptor** and **address descriptor** after each instruction is processed.

---

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:
(a - b) * (c + d) - e
Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.
Given the three-address code sequence:
t1 = a + b,
t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation